

A
PROJECT REPORT ON
“AI STUDY RECOMMENDATION SYSTEM WITH
RANDOM FOREST ALGORITHM”

SUBMITTED TO
D. Y. PATIL UNIVERSITY
A Project report submitted towards the partial fulfilment of the
requirements for the Degree of
BACHELOR OF TECHNOLOGY IN COMPUTER
ENGINEERING

SUBMITTED BY
Daksh Dewangan (SOE2022B0303046)
Dnyanesh Adkar (SOE2022B03005)
Pratik Bhole (SOE2022B0303032)
Samarth Jadhav(SOE2022B0303073)

UNDER THE GUIDANCE OF
Prof. Dipali Khairnar
DEPARTMENT OF COMPUTER SCIENCE
SCHOOL OF ENGINEERING AND TECHNOLOGY
D. Y. PATIL UNIVERSITY
A/P Ambi, MIDC Road, Talegaon Dabhade, PUNE-410507
Academic Year 2025-2026

**DEPARTMENT OF COMPUTER SCIENCE
SCHOOL OF ENGINEERING AND TECHNOLOGY**

D. Y. PATIL UNIVERSITY

A/P Ambi, MIDC Road, Talegaon Dabhade, PUNE-410507



CERTIFICATE

This is certified that the project entitled

**“AI STUDY RECOMMENDATION SYSTEM WITH RANDOM
FOREST ALGORITHM”**

Submitted By

Daksh Dewangan (SOE2022B0303046)

Dnyanesh Adkar (SOE2022B03005)

Pratik Bhole (SOE2022B0303032)

Samarth Jadhav (SOE2022B0303073)

Is a record of bonafide work carried out by them, in the partial fulfilment of the requirement for the award of Degree of Bachelor of Technology Computer Science at School of Engineering And Technology, D. Y. Patil University Pune. This work is done during the year 2025- 2026, under our guidance.

Prof. Dipali Khairnar

Project Guide

Dr. Moresh Mukhedkar

Project Coordinator

**Dr. Vivek Patil
Head of Department**

**Dr. Shweta Koparde
Dean, SOET**

External Examiner

Date:

Acknowledgements

We are profoundly grateful to **Prof. Dipali Khairnar** for his expert guidance and continuous encouragement throughout to see that this project rights its target since its commencement to its completion.

We would like to express deepest appreciation towards **Dr. Shweta Koparde**, Dean, School of Engineering & Technology, D. Y. Patil University, Pune, **Dr. Vivek Patil**, Head of Department of Computer Science and **Dr. Moresh Mukhedkar**, Project Coordinator whose invaluable guidance supported us in completing this project.

At last, we must express our sincere heartfelt gratitude to all the staff members of Information Technology Department who helped me directly or indirectly during this course of work.

Daksh Dewangan
Dnyanesh Adkar
Pratik Bhole
Samarth Jadhav

ABSTRACT

The AI Study Recommendation System is an artificial intelligence and machine learning–based application designed to provide personalized study guidance to students based on their academic performance, skills, and learning preferences. The system enables students to log in and enter relevant academic information such as subject marks, areas of interest, skill levels, and weak topics through a user-friendly interface. The collected data is processed using data preprocessing techniques such as cleaning, normalization, and feature extraction to prepare the information for analysis. The processed data is then analyzed using trained machine learning algorithms that identify patterns in the student’s performance and determine strong and weak learning areas. Based on this analysis, the system generates personalized study recommendations including suggested study topics, educational videos, and practice questions that help students focus on improving weaker subjects while strengthening their overall understanding. The system further provides a personalized study plan that includes daily or weekly learning tasks and a progress tracking dashboard that allows students to monitor their improvement over time. By integrating data analysis, intelligent recommendation techniques, and performance monitoring features into a single platform, the invention aims to enhance learning efficiency, promote targeted study practices, and support students in achieving better academic outcomes through a smart and accessible digital learning environment.

Keywords:

Core Technologies: Artificial Intelligence (AI), Machine Learning (ML), Data Preprocessing, Feature Extraction, Pattern Recognition, Predictive Analytics.

Functional Areas: Personalized Learning, Academic Performance Analysis, Automated Study Planning, Skill Gap Identification, Educational Content Recommendation, Progress Tracking.

Research Concepts: Educational Technology (EdTech), Intelligent Tutoring Systems (ITS), Recommendation Algorithms, Data-Driven Pedagogy, Adaptive Learning Environments, User-Centric Design.

INDEX

1	Introduction	10
1	Introduction	10
1.1	Problem Identified	12
1.2	Problem Statements	12
1.3	Motivation and Need	14
1.4	Project Idea	14
2	Literature Survey	16
2.1	Study Of Research Paper	17
3	Problem Definition	21
3.1	Proposed Problem Definition.....	22
3.2	Modules.....	23
3.3	Existing System.....	23
3.4	Area of Project	24
3.5	Project Scope.....	25
3.6	Details of Algorithm	25
3.7	Goal.... ..	26
3.8	Objective	27
4	Software/Hardware System Requirements	30
4	External interface requirement	31
4.1	User Interface	31
4.2	Hardware Interfaces:	31
4.3	Software Requirements	31
4.4	Software requirement specification	..31
4.5	Nonfunctional requirement	32

4.6	Performance Requirements	32
4.7	Safety Requirement	32
4.8	Software Quality Attributes	32
5	SYSTEM DESIGN	34
5.1	System Architecture	35
5.2	DataFlowDiagram.	35
5.3	UML Diagram	37
5.4	UseCaseDiagram	37
5.5	Activity Diagram	38
5.6	Sequence Diagram	39
5.7	Class Diagram	40
5.8	Flowchart Diagram	41
5.9	Mathematical Model	42
6	Software Information	43
6.1	Python	44
7	Project Plan	47
7.1	Project Scheduling	48
7.2	Project Plan	49
7.3	Gantt List	50
7.4	Responsibilities of Developer	50
8	Other Specifications	52
8.1	Advantages of System	53
8.2	Applications	53

9	Results & Discussions	55
	9.1 Results	56
	9.2 Discussion	59
10	Conculsion	61
	10.1 Conclusion	62
	10.2 Future Scope.	62
	References	64

List of Figures

5.1	System architecture	33
5.2	Data flow diagram -1	34
5.4	Use-Case Diagram	35
5.5	Activity Diagram	36
5.6	Sequence-Diagram	37
5.6.1	Training-Sequence-Diagram	38
5.6.2	Testing-Sequence-Diagram	39
5.7	Class Diagram	40
5.8	Flowchart	41
7.1	Timeline Chart	50
9.1	Home Page Interface of Smart Resume Ai Platform	56
9.2	Resume Upload and ATS Analysis Interface	57
9.3	ATS Resume Score and Analysis Dashboard	57
9.4	Resume Strength and Improvement Suggestions	58
9.5	Recommended learning Resources and AI Interview Option	58
9.6	AI Voice Mock Interview Interface	59

CHAPTER 1

INTRODUCTION

INTRODUCTION

The way we learn today is vastly different from how it was a generation ago. With the internet, we have access to countless online courses, digital libraries, and learning websites. While having so much information at our fingertips is a great advantage, it also creates a new problem: it's hard to know where to start. Many students feel lost in this sea of information, unsure which resources are best for them or what they should focus on. The traditional, one-size-fits-all approach to education doesn't work well when everyone learns at a different pace and has different strengths.

This is where Artificial Intelligence (AI) and Machine Learning (ML) can make a real difference. These technologies are changing the field of educational technology by making it possible to create learning systems that adapt to each student. The current trend is to use data from a student's learning activities to offer them a personalized path. For example, some platforms can suggest courses or predict grades based on a student's progress. Our project, the "AI Study Recommendation System," fits right into this area. We use the Random Forest algorithm, a well-known machine learning technique, to power our system. The application is built with a Flask backend and a React frontend to provide a complete and interactive experience for the user.

1.1 Problem Identified

The main problem we are addressing is that students often struggle to find the right study materials that match their learning needs. They have access to more resources than ever before, but this abundance can be overwhelming rather than helpful. This leads to inefficient studying, where students might waste time on topics they already know or get stuck on difficult subjects without proper guidance. Most current learning platforms simply provide a library of content without offering any real direction. These systems are not very effective because they don't consider the individual student's academic background, their learning speed, or the specific subjects they find challenging. This can result in a frustrating learning experience and prevent students from reaching their full potential.

1.2 Problem Statements

This project aims to solve the problem of inefficient learning by creating an intelligent system that offers personalized study recommendations. The system will analyze a student's academic profile to

suggest specific topics, learning resources, and study schedules designed to improve their academic performance and make their study time more effective.

Statement of scope:

The scope of this project is to design, develop, and implement a fully functional AI-powered Study Recommendation System. The system will be delivered as a web-based application accessible through standard web browsers.

The primary functionalities within the scope of this project include:

- a) **User Management:** The system will provide secure user registration and authentication, allowing students to create and manage their personal accounts.
- b) **Student Profiling:** It will include a feature for students to input and update their academic information, including subject marks, skills, and personal interests.
- c) **AI-Based Analysis:** The core of the system will involve the implementation of a Random Forest algorithm to analyze student data and accurately identify their academic strengths and weaknesses.
- d) **Personalized Recommendations:** Based on the AI analysis, the system will recommend tailored study topics, relevant educational resources from a predefined catalog, and practice questions.
- e) **Study Planning and Tracking:** The application will generate personalized daily and weekly study plans and feature a dashboard to help students track their progress over time.
- f) **Performance Prediction and Adaptive Learning:** The system will incorporate a predictive model to forecast student performance and an adaptive module to provide questions tailored to the student's level of understanding.

AI Tutoring: An integrated AI chatbot will be included to provide on-demand assistance and answer student queries.

The project is limited to the following boundaries and will not include:

The creation of original educational content; the system will recommend existing, curated resources.

1.3 Motivation and Need

The motivation behind this project is the need to make education a more personal and effective experience for every student. In today's competitive world, students need guidance that is tailored to their unique abilities and challenges. An AI-powered recommendation system is important because it directly tackles the limitations of the traditional, generalized approach to education. The real-world benefit of such a system is significant. It can help lower student stress, build their confidence, and encourage a more engaged attitude towards their studies. By offering clear and personalized guidance, this project can empower students to manage their learning more effectively, making education a more productive and enjoyable process.

1.4 Project Idea

Our idea is to build a complete AI Study Recommendation System as a web application. The main goal is to use a machine learning model to create a personalized learning journey for each user. The system will start by collecting key information from the student, such as their grades in different subjects, their existing skills, and their personal interests. This data will be used to build a detailed profile for each student.

This profile is then fed into a Random Forest algorithm. We chose this specific algorithm because it is known for being accurate and reliable. The algorithm will analyze the student's profile to figure out which topics they are good at and which ones they struggle with. Based on this analysis, the system's recommendation engine will search through our collection of study materials—including articles, videos, and practice problems—and suggest the best resources to help the student improve. To provide more structure, the system will also create a personalized study planner with daily and weekly tasks. This all-around approach gives the student a clear and customized plan to follow for their studies.

CHAPTER 2

LITERATURE SURVEY

2.1 STUDY OF RESEARCH PAPER

A thorough review of existing literature was conducted to understand the current state of research in the field of educational recommendation systems, student performance prediction, and the application of machine learning in personalized learning. The following research papers provide the foundation upon which this project is built.

1. Paper Name: “A Recommendation System for E-learning based on Collaborative Filtering and learning Style”

Author: A. M. El-Sattar

Description: Published in the International Journal of Computer Science and Information Security (IJCSIS) in 2017. This paper proposes an e-learning recommendation system that uses collaborative filtering combined with the Felder-Silverman learning styles model. The system groups learners based on their learning styles and then generates recommendations from users with similar preferences. The findings showed that incorporating learning styles improved the accuracy and user satisfaction of the recommendations. The primary limitation is its reliance on explicit user ratings and a predefined learning style model, which may not capture the full complexity of a student's needs.

Output: The system groups learners based on their learning styles and then generates recommendations from users with similar preferences. The findings showed that incorporating learning styles improved the accuracy and user satisfaction of the recommendations.

Paper Title: A Recommendation System for E-learning based on Collaborative Filtering and Learning Style

Journal/Year: International Journal of Computer Science and Information Security (IJCSIS), 2017.

Methodology: The system integrates Collaborative Filtering with the Felder-Silverman Learning Styles Model to group students by preference and style.

Key Finding: Combining these two methods results in higher accuracy and increased user satisfaction compared to standard recommendation engines.

Constraint: The model currently depends on explicit user ratings, which can be a limitation if students are inactive in rating content.

Research Output: Proposed an e-learning recommendation framework that integrates collaborative filtering with the Felder-Silverman learning styles model to group similar learners. Demonstrated through experimental findings that incorporating individual learning styles significantly improves both recommendation accuracy and overall user satisfaction. Identified a primary limitation in the system's reliance on explicit user ratings and predefined models, which may fail to capture the full complexity of dynamic student needs.

2. Paper Name: "Student Performance Prediction Using Random Forest"**Author:** S. K. S. Tyagi and Q. V. Dang

Description: Used the Random Forest algorithm to predict how well students would perform. They based their predictions on factors like student demographics, past grades, and online activity. Their research showed that Random Forest was very good at figuring out which factors were most important for student success and made accurate predictions. The main takeaway for us was that while the model was good at prediction, it didn't use those predictions to offer any guidance or recommendations to the students.

Methodology: The researchers applied the Random Forest algorithm, an ensemble learning method, to forecast student outcomes

Data Features: Predictions were synthesized from a multi-dimensional dataset including demographics, historical academic records, and real-time online engagement metrics

Key Strength: The model excelled at *feature importance ranking*, identifying exactly which variables (like online activity vs. past grades) most heavily influenced student success

Critical Gap: The study noted a lack of actionable intervention, as the system provided accurate performance forecasts but failed to translate those data points into automated guidance or student recommendations

Research Output: Implemented a Random Forest-based predictive model that utilized student demographics, academic history, and online activity to accurately forecast performance outcomes. Demonstrated the algorithm's high efficacy in identifying and ranking the most significant variables contributing to student success within a digital learning environment. Highlighted a functional gap in current predictive systems where high-accuracy forecasts are not yet integrated with automated recommendation or guidance mechanisms for the students.

3. Paper Name: “A Hybrid Approach for Course Recommendation System in E-learning”**Author:** P. C. Behera and S. K. Rath

Description: Presented a system that combined two different methods: content-based and collaborative filtering. This hybrid system recommended courses by looking at what a student had studied before and what similar students had chosen. This approach helped solve the "cold-start" problem, where a system can't make recommendations for new users. We noted that this system was good at recommending entire courses but didn't offer suggestions for more specific topics within those courses.

Methodology: The authors implemented a hybrid recommendation engine that integrates content-based filtering* with collaborative filtering to leverage the strengths of both.

Technical Advantage: By utilizing this dual-method approach, the system effectively addresses the *"cold-start" problem*, ensuring that new users receive relevant suggestions even with limited historical data

Paretional Scope: The system analyzes a student's personal study history alongside the preferences of a similar peer group to generate course-level suggestions

Limitation: While the hybrid model is proficient at recommending entire courses, it lacks the granularity required to suggest specific sub-topics or modular lessons within those courses

Research Output: Developed a hybrid recommendation system that combines content-based and collaborative filtering to provide accurate course suggestions based on both individual history and peer behaviour. Successfully mitigated the "cold-start" problem, enabling the system to provide meaningful educational recommendations for new users with minimal initial data. Identified a need for greater granularity in future models to transition from broad, course-level recommendations to specific topic-based suggestions within the curriculum.

4. Paper Name: "An Adaptive E-learning System Based on Student's Learning Style"**Author:** M. A. F. Al-Ahdal and M. A. Al-Dhaqm

Description: Their work focused on creating a system that changed how content was presented based on a student's learning style, which was determined through a questionnaire. They found that this adaptation kept students more engaged. A clear area for improvement would be to figure out a student's learning style automatically based on their behavior, rather than having them fill out a form.

Methodology: The research focused on the development of a system that dynamically adapts the presentation of educational content according to the specific learning styles of individual students.

Assessment Tool: Student learning styles were initially identified using a traditional questionnaire-based method to categorize user preferences.

Key Finding: The study demonstrated that personalizing the delivery of content based on these styles led to higher levels of student engagement compared to non-adaptive systems.

Future Direction: The authors identified a significant opportunity for improvement by suggesting a shift from manual forms to automated learning style detection based on real-time student behaviour.

Research Output: Developed an adaptive e-learning system that modifies content presentation based on learning styles to enhance the student experience. Validated that adapting educational materials to individual student profiles significantly increases and sustains learner engagement. Proposed a technical transition from manual questionnaire-based style identification to automated, behaviour-driven detection for future implementations.

5. Paper Name: "Educational Data Mining: A Survey and a Data Mining-Based Analysis of Recent Works"

Author: C. Romero and S. Ventura

Description: Provided a broad overview of the field. It covered the different ways data mining can be used in education, such as for prediction and clustering. The paper was a great resource for understanding the various techniques available. It also pointed out a common issue in early systems: they were often good at finding insights but lacked a user-friendly way to share those insights with students.

Field Overview: The authors provided an extensive survey of Educational Data Mining (EDM), serving as a foundational resource for the various techniques available in the domain.

Functional Applications: The research categorized the use of data mining in education into several key areas, most notably for student performance prediction and clustering analysis.

Technique Mapping: The paper detailed how these various mathematical and statistical techniques can be applied to educational datasets to extract meaningful patterns.

Usability Gap: A significant critique identified in the work is that early EDM systems, while technically proficient at uncovering insights, often failed to provide a user-friendly interface or mechanism to communicate those findings directly to the students.

Research Output: Provided a comprehensive survey of Educational Data Mining techniques, specifically highlighting the application of prediction and clustering within academic environments. Established a broad resource for understanding the diverse methodologies available to extract actionable insights from educational datasets. Identified a critical usability flaw in early systems where complex data insights lacked an accessible or user-friendly method for sharing results with students.

Table1:
Literature review

Sr. no	Paper Name	Author Name	Description	Remark
1.	A Recommendation System for E-learning based on Collaborative Filtering and Learning Style	A. M. El-Sattar	This paper proposes an e-learning recommendation system using collaborative filtering combined with the Felder-Silverman learning style model. It groups learners based on learning styles and provides personalized recommendations. The system improves accuracy and user satisfaction.	Relies on explicit user ratings and predefined learning styles, which may not fully capture individual learning behavior.
2.	Student Performance Prediction Using Random Forest	S. K. S. Tyagi, Q. V. Dang	This study uses the Random Forest algorithm to predict student performance based on demographics, past grades, and activity data. It effectively identifies key factors influencing student success and provides accurate predictions.	Focuses only on prediction and does not provide recommendations for improvement.

3.	A Hybrid Approach for Course Recommendation System in E-learning	P. C. Behera, S. K. Rath	This paper presents a hybrid recommendation system combining content-based and collaborative filtering. It recommends courses based on user history and similar users, solving the cold-start problem.	Limited to course-level recommendations and lacks topic-specific guidance.
4.	An Adaptive E-learning System Based on Student's Learning Style	M. A. F. Al-Ahdal, M. A. Al-Dhaqm	This system adapts learning content based on student learning styles identified through questionnaires. It enhances engagement and improves learning outcomes.	Requires manual input for learning style detection instead of automatic analysis.
5.	Educational Data Mining: A Survey and a Data Mining-Based Analysis of Recent Works	C. Romero, S. Ventura	This paper provides an overview of educational data mining techniques such as prediction and clustering. It highlights the importance of data-driven decision-making in education systems.	Focuses on analysis but lacks practical implementation for real-time student support.

CHAPTER 3

PROBLEM DEFINITION

3.1 Proposed Problem Definition

The central problem this project addresses is the inherent inefficiency and lack of personalization in conventional e-learning platforms. These systems typically deliver static content, failing to adapt to a student's unique learning trajectory, cognitive strengths, or knowledge gaps. This results in a suboptimal learning experience where students are either overwhelmed by irrelevant information or left without adequate support in challenging areas. The proposed solution is a data-driven, algorithmic system designed to overcome these limitations by leveraging machine learning to model student competency and deliver truly personalized, actionable study recommendations.

3.2 Modules

The system's architecture is modular, comprising several interconnected components that work in concert to deliver a seamless user experience.

User Authentication and Profiling Module: This module provides secure, token-based user authentication using JSON Web Tokens (JWT). It is responsible for the creation and persistent storage of detailed student profiles in a MongoDB database, capturing multi-dimensional data including subject-wise academic scores, self-assessed skills, and declared areas of interest.

AI-Powered Analysis Engine: This core module serves as the analytical backbone of the system. It ingests the structured data from the student's profile and employs a trained Random Forest classifier. Its primary function is to perform a diagnostic analysis, classifying each topic within a subject domain as either a "strength" or a "weakness" for that specific student.

Hybrid Recommendation Engine: This module translates the output of the analysis engine into concrete recommendations. It utilizes a two-pronged Natural Language Processing (NLP) strategy. Initially, TF-IDF is used for rapid keyword-based indexing of the resource catalog. Subsequently, Sentence-BERT (SBERT) provides deep semantic understanding, allowing the engine to rank resources based on contextual relevance to the student's identified weak topics, ensuring recommendations are not just topically related but also conceptually appropriate.

Study Planner and Progress Visualization Module: To promote structured learning, this module algorithmically generates personalized daily and weekly study schedules based on the recommendations. It also features a dynamic dashboard, which fetches progress data via API calls and

uses libraries like Recharts to render visualizations of student performance and task completion over time.

Adaptive Learning and Tutoring Module: This module fosters an interactive learning environment. It includes an algorithm to dynamically generate practice questions whose difficulty level is adjusted based on the student's ongoing performance. It also integrates with the OpenAI API to power a chatbot tutor, providing contextual and immediate responses to student queries

3.3 Existing System

Existing systems in the e-learning landscape, such as massive open online course (MOOC) platforms like Coursera or edX and traditional Learning Management Systems (LMS) like Moodle, primarily operate as content repositories. Their core technology is centered around content delivery networks and database management for storing courses and user data. While some employ basic recommendation algorithms (e.g., "students who took this course also took..."), these are typically based on simple collaborative filtering and do not perform a deep analysis of an individual user's academic competency.

Disadvantages of Existing System:

- a) **Lack of Granular Personalization:** Recommendations are often at the course level and are based on popularity or collaborative data, not on an individual's specific knowledge gaps.
- b) **High Cognitive Load on the User:** The responsibility for identifying relevant topics and resources falls almost entirely on the student, which can be inefficient and overwhelming.
- c) **Static Difficulty and Learning Path:** The systems do not dynamically adjust the difficulty of content or the learning trajectory based on a user's real-time performance.
- d) **Limited Feedback Loop:** They lack a mechanism to diagnose why a student is struggling and then provide a targeted, actionable plan for remediation.

3.4 Proposed System

This project is fundamentally an interdisciplinary endeavor that integrates concepts from Artificial Intelligence (AI), specifically in the sub-fields of Machine Learning (ML) and Natural Language Processing (NLP), with Full-Stack Web Development. Its real-world application lies squarely in the

Educational Technology (EdTech) sector, providing a blueprint for next-generation adaptive learning platforms that can be deployed in academic institutions or used by individuals for self-paced learning.

3.5 Project Scope

The scope of this project is the end-to-end design, development, and local deployment of a functional prototype of the AI Study Recommendation System. This includes creating the Flask-based RESTful API for all backend logic, designing the MongoDB database schema, and building a responsive and interactive frontend using React. The project's boundaries are set to demonstrating the core functionality on a curated dataset and resource catalog. Future scalability is a key consideration in the design; the modular architecture allows for the easy integration of new machine learning models, expansion to additional subject domains, and potential integration with external APIs or institutional databases.

3.6 Details of algorithms

The intelligence of the system is driven by a carefully selected set of algorithms:

Random Forest: This is an ensemble learning method that functions by constructing a large number of decision trees during training. Each tree is trained on a random subset of the data (a technique called bootstrap aggregating or "bagging"), and at each node, only a random subset of features is considered for the split. This dual randomization strategy makes the model highly robust against overfitting. For classification, the final prediction is determined by a majority vote from all trees. We chose Random Forest because of its high accuracy, its ability to handle both numerical (grades) and categorical (interests) data seamlessly, and its intrinsic capability to calculate feature importance, which can provide insights into what factors most influence a student's performance.

TF-IDF (Term Frequency-Inverse Document Frequency): This algorithm evaluates the relevance of a word within a document relative to a collection of documents (a corpus). The Term Frequency (TF) measures how often a word appears in a document, while the Inverse Document Frequency (IDF) measures how rare or common that word is across all documents. The TF-IDF score is the product of these two values. We use it for efficient, large-scale keyword extraction to create an initial set of candidate resources from our catalog.

Sentence-BERT (SBERT): To overcome the context-blind nature of TF-IDF, we employ SBERT. This model modifies the standard BERT architecture by using a siamese network structure, which processes two sentences in parallel to generate fixed-size, semantically rich vector embeddings for each. The cosine similarity between these vectors provides a highly accurate measure of their semantic relationship. We chose SBERT for its state-of-the-art performance in semantic similarity tasks, allowing our system to recommend resources that are conceptually related to a student's weak areas, even if they don't share the exact same keywords.

3.7 Goal

The primary goal of this project is to engineer an intelligent, closed-loop learning system that significantly enhances student academic performance by providing continuous, personalized, and data-driven guidance throughout their learning journey.

3.8 Objective

- a) To design and implement a secure, scalable backend with a RESTful API for managing user data and serving recommendations.
- b) To train and integrate a Random Forest classifier capable of accurately identifying a student's academic strengths and weaknesses from their profile data.
- c) To develop a hybrid recommendation engine combining TF-IDF and Sentence-BERT to rank and suggest relevant learning resources with high precision.
- d) To build a responsive frontend with a user-friendly dashboard for visualizing progress and interacting with the study planner.
- e) To implement an adaptive questioning module and an AI-powered chatbot to create an interactive and supportive learning environment.
- f) To conduct a systematic evaluation of the system's recommendation quality and its overall effectiveness as a personalized learning tool.

CHAPTER 4

HARDWARE AND SOFTWARE REQUIREMENTS

4. EXTERNAL INTERFACE REQUIREMENT

4.1 User Interface

Users will interact with the system through a simple and clean website. The user interface is designed to be straightforward and easy to use. Key features include:

Login and Signup: Standard pages for users to securely log in or create a new account.

Profile Page: A simple form where students can enter their grades, skills, and interests.

Dashboard: This is the main screen where users will see their personalized recommendations, such as what topics to study next and links to helpful resources. It will also show their study schedule.

Progress Charts: The dashboard will use graphs and charts to give students a quick visual update on their progress.

Interactive Tools: The site will also have a chat window for the AI tutor and a section for students to answer practice questions.

4.2 Hardware Interfaces:

Our system is a web application, so it doesn't need any special hardware to run.

For Users: Anyone with a standard computer, laptop, or tablet that has a modern web browser and an internet connection can use the system.

For the Server: The backend of the application, which runs the AI models, is designed to run on a standard server. For our development, we used a local machine with a multi-core processor and at least 8 GB of RAM to make sure it could handle all the processing. For training more complex models, a GPU would be beneficial.

Major Constraints

Hardware:

- a) RAM: 8GB
- b) Key Board: Standard Windows Keyboard

The camera catches the picture and sends the processor of the PC which comprises of a 32-bit memory card introduced with OpenCV which helps in picture handling. On the off chance that the sign passes the boundary of a bunch of persistent casings with EAR not as much as limit esteem, it will naturally make the alert blare and the speed of the vehicle gets diminished. In any case that sign is dismissed and the following sign is handled.

4.3 Software Requirements

The project was built using a specific set of software and tools.

1. Operating System: The system can run on Windows, macOS, or Linux.
2. IDE/Tools:
 - a) Visual Studio Code (VS Code): Used as the primary code editor for both frontend and backend development.
 - b) Git: Used for version control to track changes in the code.
3. Programming Languages
 - a) Backend: Python (version 3.8 or higher).
 - b) Frontend: JavaScript (ES6+).
4. Key Libraries and Frameworks:

Backend:

- a) Flask: A lightweight web framework used to build our backend API.
- b) Pandas & NumPy: Essential libraries for data manipulation, cleaning, and numerical operations, used to process the student data before feeding it into the models.
- c) Scikit-learn: Used for implementing the Random Forest algorithm and other traditional machine learning tasks.
- d) TensorFlow: A comprehensive machine learning framework used for building, training, and deploying deep learning models.

- e) Sentence-Transformers: A library built on TensorFlow/PyTorch to generate semantic embeddings for the recommendation engine.
- f) PyMongo: Used to connect the Python application with the MongoDB database.

Frontend:

- a) React.js: A JavaScript library for building the user interface.
- b) Vite: A modern build tool for frontend development.
- c) Axios: Used for making API calls from the frontend to the backend.
- d) Recharts: A library for creating the data visualization charts on the dashboard.

5. Database:

- MongoDB: A NoSQL database used to store user profiles, progress, and other application data.

4.4 Software requirement Specification Purpose and Scope of Document

The main purpose of this software is to give students a personalized learning experience. The system is designed to analyze their academic performance and suggest study materials that are right for them. The scope of the project includes everything from user signup and profile creation to getting smart recommendations, following a study plan, and tracking progress.

4.5 NONFUNCTIONAL REQUIREMENT

The AI Study Recommendation System must satisfy several non-functional requirements to ensure efficiency and usability. The system should provide fast performance by generating recommendations within a few seconds and handling multiple users simultaneously without delays. It must be scalable to support a growing number of students and large datasets. The interface should be user-friendly, allowing easy navigation with minimal training. The system should be reliable and available 24/7 with minimal downtime and accurate outputs. Strong security measures such as authentication,

authorization, and data encryption must be implemented to protect sensitive student data. Additionally, the system should ensure data privacy by preventing unauthorized access. It must be maintainable with modular code for easy updates and improvements. The application should be portable across different devices and browsers. High availability and backup mechanisms are required to prevent data loss. The system should maintain high accuracy in recommendations and continuously improve using feedback. Lastly, it should be flexible enough to allow future enhancements and integration with other platforms.

4.6 Performance Requirements

The system needs to feel fast and responsive. When a user clicks a button or requests information, the website should load quickly. Our goal was to have most data load in under a second to ensure a smooth and frustration-free experience.

4.7 Safety Requirement

Protecting user data is a top priority. All user passwords are encrypted (hashed) so they cannot be read. The system is also built to handle errors gracefully, so if something goes wrong, it won't crash or lose data.

4.8 Software Quality Attributes

- a) **Maintainability:** We wrote the code in a clean and organized way, with comments to explain what each part does. Because the frontend and backend are separate, it will be easier to make updates or add new features in the future without breaking anything.
- b) **Reliability:** The system should be dependable and work as expected. We included error handling in our code to make sure it remains stable even if there are unexpected issues.
- c) **User Friendliness:** The website is designed to be easy to use. A new user should be able to understand how it works without needing a lot of instructions.
- d) **Security:** We used modern security practices to protect user accounts. All user data is kept private, and communication between the user's browser and our server is encrypted.
- e) **Testability:** The way we organized our project makes it easy to test. We can test individual parts of the backend and frontend separately to make sure everything is working correctly.

CHAPTER 5

SYSTEM DESIGN

5.1 System architecture of AI study recommendation system

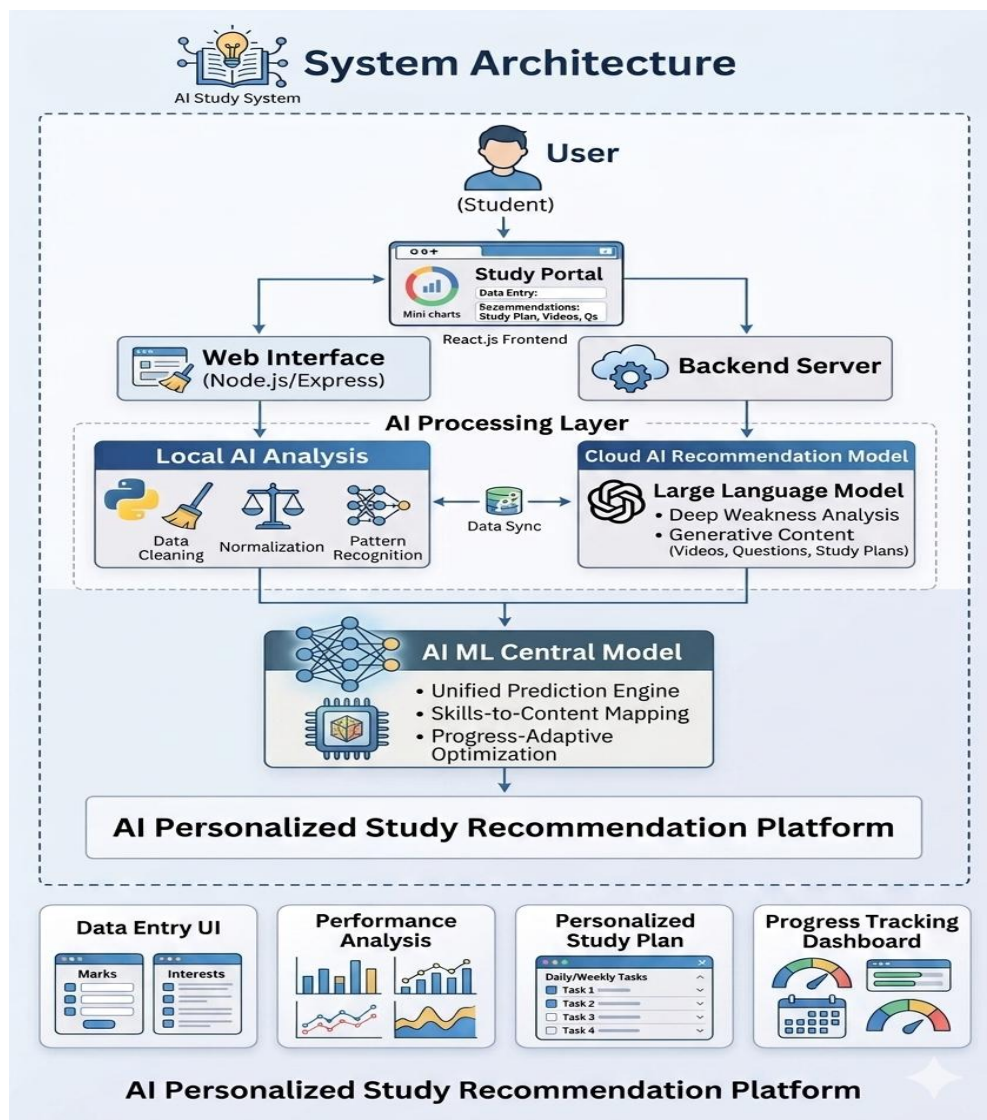


Figure 5.1: System architecture

A student interacts with a React frontend, which sends requests to a Flask backend API. The backend manages core features like authentication, study planning, recommendations, progress tracking, prediction, and chatbot.

It connects with an AI/ML layer for intelligent processing (like NLP and adaptive learning) and may use external AI APIs. All data (users, progress, plans) is stored in a database.

Overall, it shows how frontend, backend, AI, and database work together to deliver personalized learning.

5.2 Data flow diagram

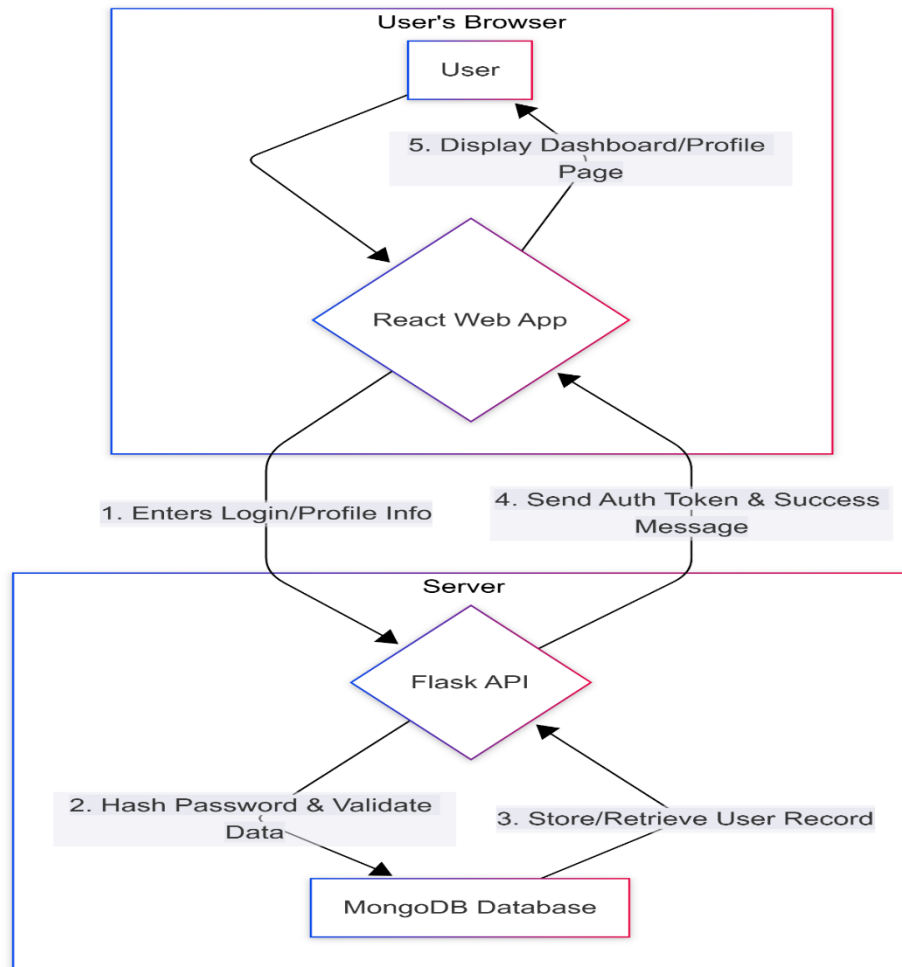


Figure 5.2: Data flow diagram 1

The user enters login/profile details in the React web app, which sends the data to the Flask API. The backend hashes and validates the password and interacts with the MongoDB database to store or retrieve user records.

After successful validation, the server sends an authentication token and success message, and the frontend displays the dashboard/profile page to the user.

5.3 UML DIAGRAM

Following diagram shows the UML Diagram of this system.

1. Usecase Diagram
2. Activity Diagram
3. Sequence Diagram
4. Class Diagram

5.4 USE CASE DIAGRAM

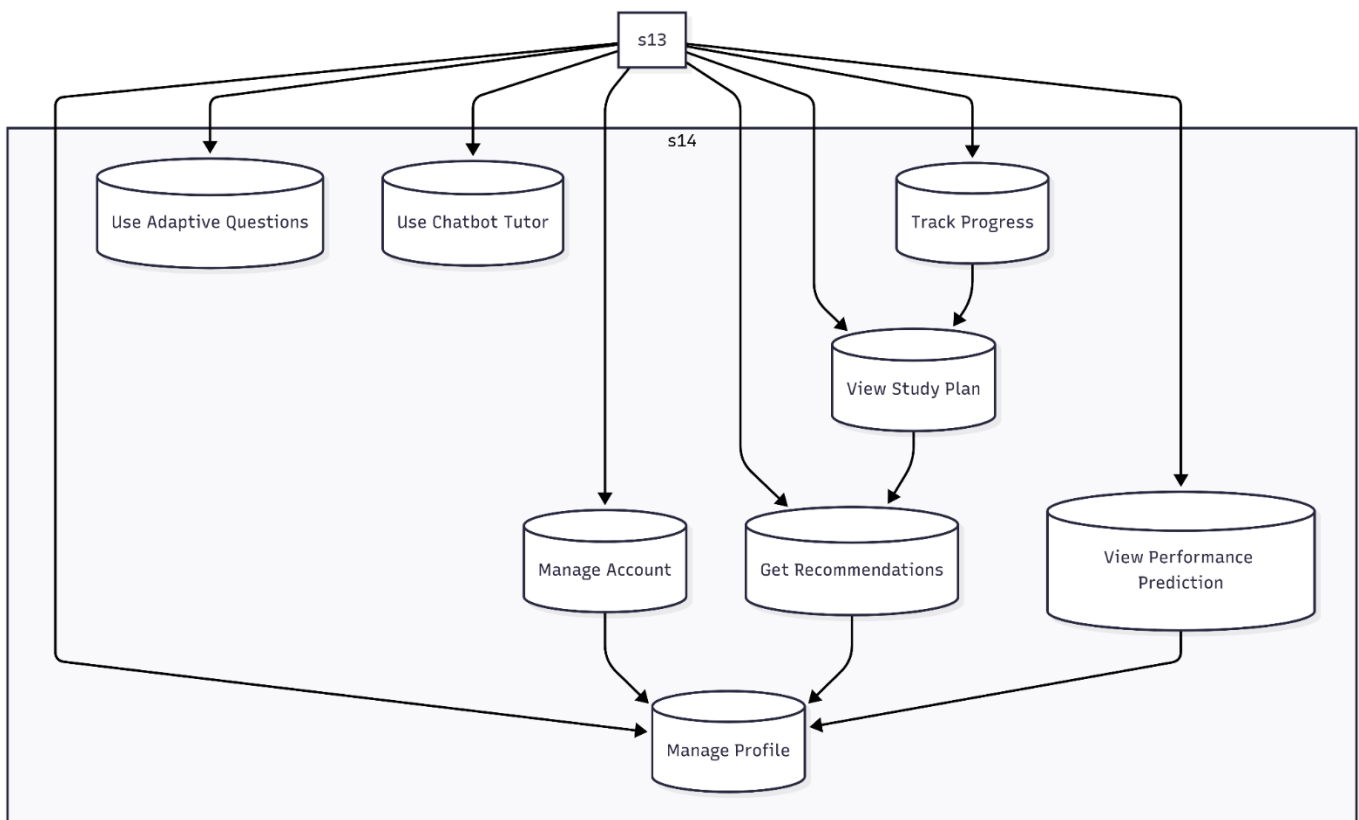


Figure 5.4: Use Case Diagram

The student (S13) can perform multiple actions like using adaptive questions, interacting with a chatbot tutor, tracking progress, and viewing study plans. Based on this, the system provides recommendations and performance predictions.

The user can also manage their account and profile, while all features are interconnected to deliver a personalized learning experience.

5.5 ACTIVITY DIAGRAM

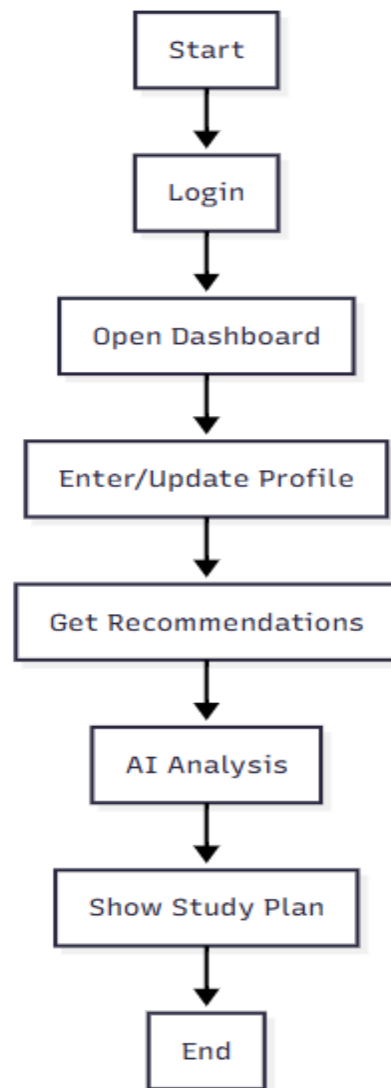


Figure 5.5: Activity Diagram

The process starts with login, followed by opening the dashboard. The user then enters or updates their profile, after which the system provides recommendations. These are processed through AI analysis, and finally, a personalized study plan is displayed before ending the process.

5.6 SEQUENCE DIAGRAM

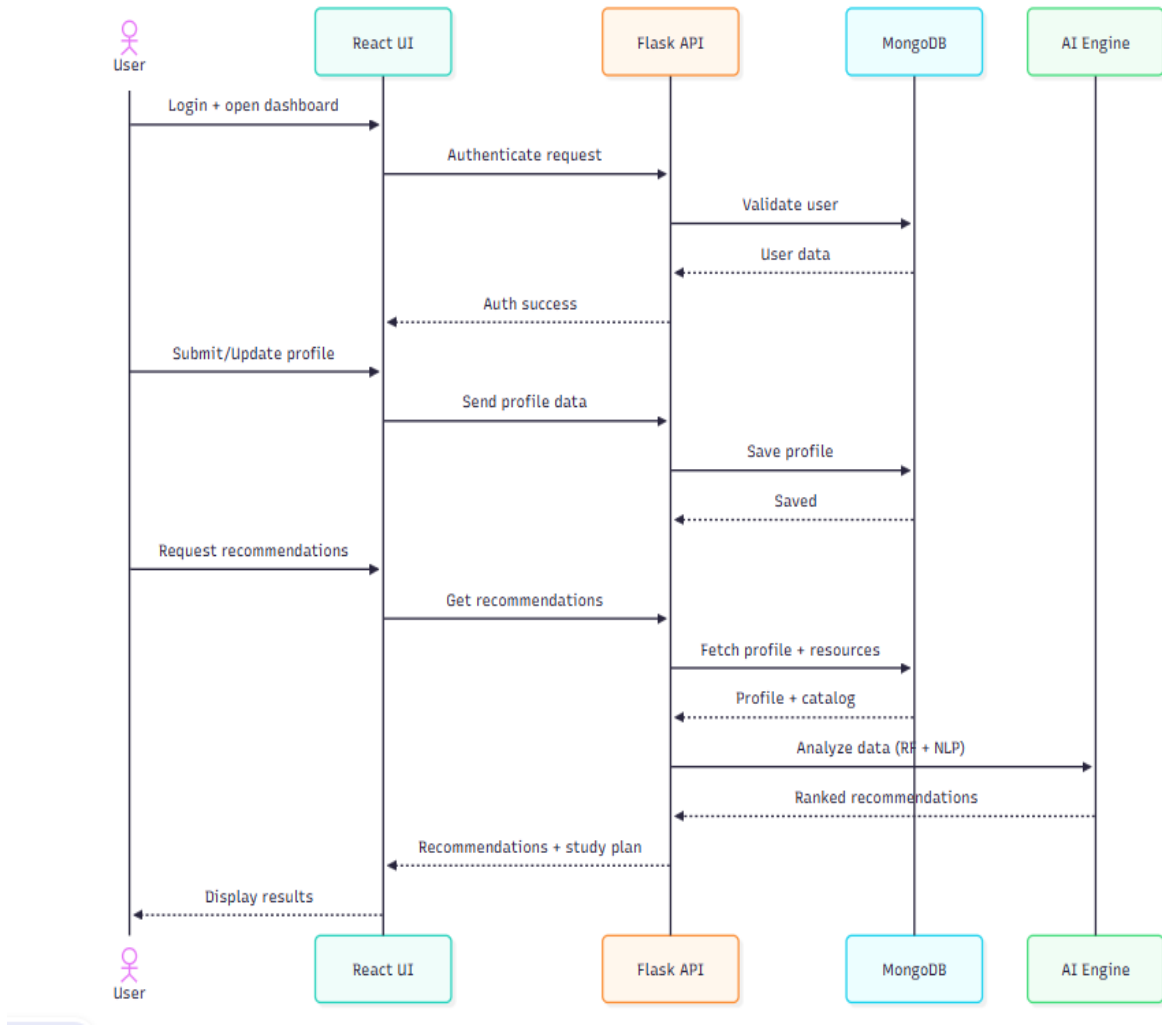


Figure 5.6: Sequence Diagram

This diagram shows the sequence flow of an AI-based study recommendation system. The user interacts with the React UI to login, update profile, and request recommendations. The Flask API handles authentication and data processing, while MongoDB stores and retrieves user data. The system then sends data to the AI engine, which analyzes it and generates ranked recommendations and a study plan. Finally, the results are returned to the frontend and displayed to the user.

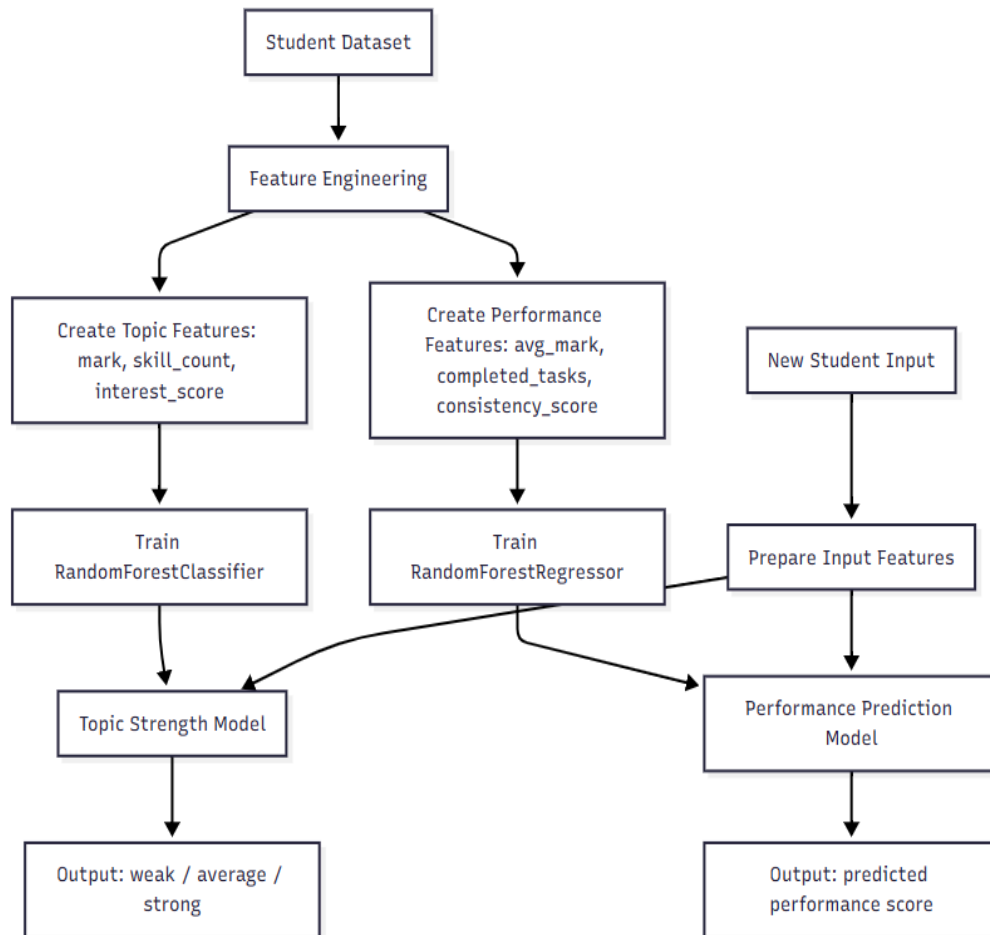
a) Training Diagram

Figure 5.7: Training Sequence Diagram

This diagram shows a machine learning pipeline for student performance analysis. The system uses a student dataset and performs feature engineering to create topic and performance features. It then trains two models: a classifier to predict topic strength (weak/average/strong) and a regressor to predict performance score. For new student data, the system processes inputs and generates predicted performance and insights.

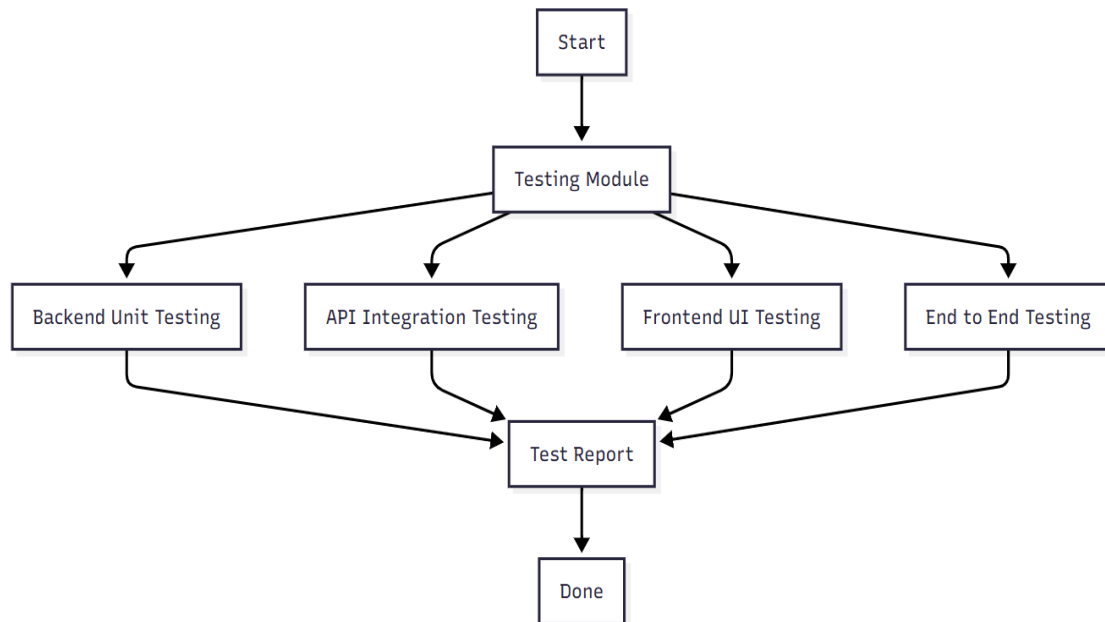
b) Testing Diagram

Figure 5.8: Testing Sequence Diagram

This diagram shows the software testing workflow.

After starting, the system performs different tests: backend unit testing, API integration testing, frontend UI testing, and end-to-end testing. All results are combined into a test report, and then the process is completed.

5.7 CLASS DIAGRAM

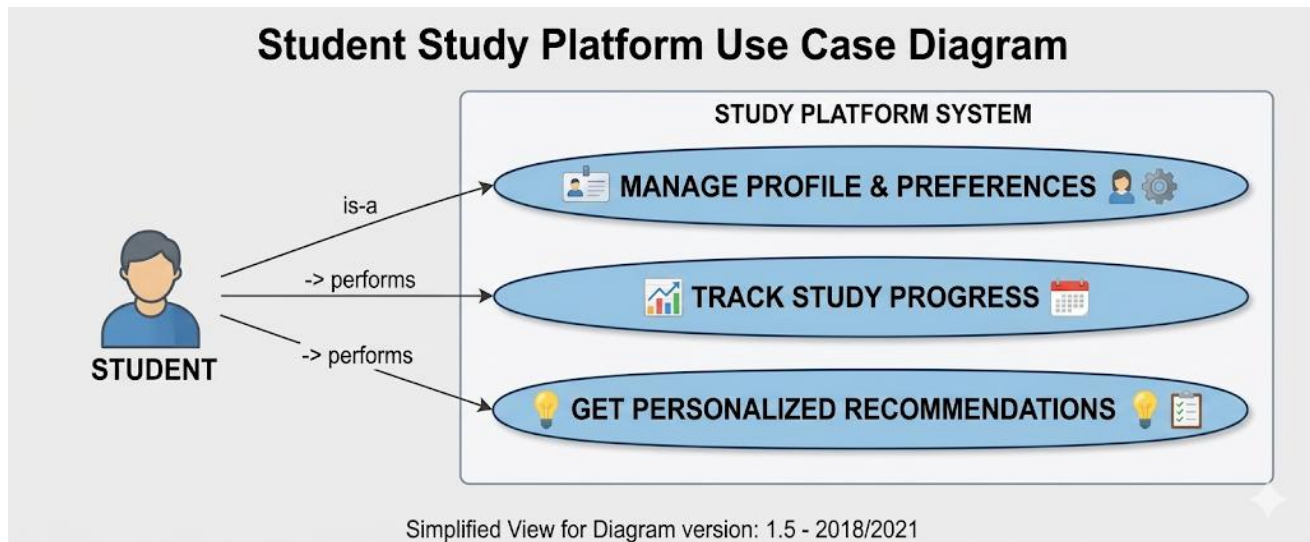


Figure 5.9: Class Diagram

This diagram shows the use case of a student in a study platform system.

The student interacts with the system to perform key actions like managing profile and preferences, tracking study progress, and getting personalized recommendations. It highlights the main functionalities available to the user in the platform.

5.8 FLOWCHART DIAGRAM

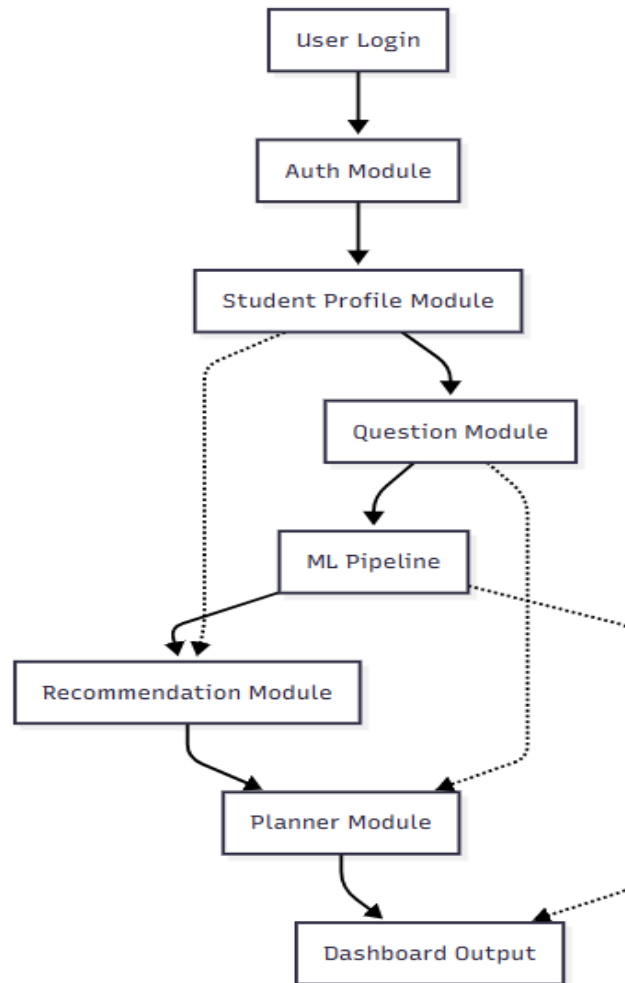


Figure 5.10: Flowchart Diagram

This diagram shows the module flow of an AI-based study system. The process starts with user login and authentication, followed by the student profile module. Then, the question module and ML pipeline process the data to generate insights. Based on this, the system provides recommendations, which are used by the planner module to create a study plan. Finally, all results are displayed in the dashboard output for the user.

5.9 MATHEMATICAL MODEL

1. Topic Strength Classification

Input features:

- $X_1 = \text{mark}$ (student score in a topic)
- $X_2 = \text{skill_count}$ (number of related skills mastered)
- $X_3 = \text{interest_score}$ (student interest level)

Prediction function:

$$\hat{Y}_{topic} = f_{cls}(X_1, X_2, X_3)$$

Description:

This function classifies a student's understanding of a topic into categories:

$$\hat{Y}_{topic} \in \{\text{weak, average, strong}\}$$

It helps identify which topics need more focus.

2. Performance Score Prediction

Input features:

- $Z_1 = \text{avg_mark}$ (overall academic performance)
- $Z_2 = \text{completed_tasks}$ (learning activity level)
- $Z_3 = \text{consistency_score}$ (regularity in study)

Prediction function:

$$\hat{Y}_{perf} = f_{reg}(Z_1, Z_2, Z_3)$$

Description:

This regression function predicts a **continuous performance score**, indicating the student's overall capability.

3. Recommendation Priority Score

$$R_t = \alpha(1 - X_{1_{norm}}) + \beta(1 - X_{2_{norm}}) + \gamma(X_{3_{norm}})$$

Description:

- Lower marks and skills increase priority $\rightarrow (1 - X)$
- Higher interest increases priority $\rightarrow X_3$
- α, β, γ are weights controlling importance

Constraints:

$$\alpha + \beta + \gamma = 1, \quad \alpha, \beta, \gamma \geq 0$$

Topics with higher R_t are recommended first.

4. Adaptive Difficulty Rule

$$\text{Difficulty} = \begin{cases} \text{Easy,} & \hat{Y}_{perf} < \tau_1 \\ \text{Medium,} & \tau_1 \leq \hat{Y}_{perf} < \tau_2 \\ \text{Hard,} & \hat{Y}_{perf} \geq \tau_2 \end{cases}$$

Description:

This rule adjusts the difficulty level of questions based on predicted performance.

5. Objective Function

Goal:

Maximize learning effectiveness by:

- Improving weak topics
- Increasing overall performance score

Description:

The system dynamically adapts recommendations to optimize student progress.

Step1: user will login to the system.

Step2: user will complete/select student profile details (skills, interests, goals).

Step3: user will submit learning/progress data for analysis.

Step4: System will apply ML models (Topic Strength Classification and Performance Prediction).

Step5: System will generate personalized recommendations and study plan.

OUTPUT: Display predicted performance, topic strength (weak/average/strong), and recommended study actions on the dashboard.

CHAPTER 6

SOFTWARE INFORMATION

6.1 Software Overview

The AI Study Recommendation System is built using a combination of web, data, and machine learning technologies. The software stack includes tools for frontend development, backend API handling, data processing, and prediction modeling.

These technologies were selected because they are reliable, well-documented, and suitable for academic as well as practical implementation. They also integrate smoothly with each other, which helped in building a complete system where user input, model processing, and recommendation output work together in one workflow.

6.2 Programming Language

The main programming language used in this project is Python for backend and machine learning tasks. Python is widely used in AI projects because its syntax is simple, readable, and easy to maintain. This was helpful for implementing modules such as authentication, recommendation generation, prediction, and progress tracking.

Python also provides strong support for data analysis and machine learning through its library ecosystem. For frontend development, JavaScript is used with React to create an interactive user interface. This combination of Python and JavaScript supports both intelligent processing and user-friendly design.

6.3 Libraries and Frameworks

The following libraries and frameworks are important in this project:

a) Flask (Backend/API)

Flask is used to build REST APIs and manage backend routing. It handles core operations such as user login, profile updates, recommendation requests, and prediction responses.

b) React (Frontend)

React is used to design the client-side interface. It supports reusable components and dynamic page updates, which improves user interaction and dashboard usability.

c) Pandas and NumPy (Data Processing)

Pandas is used for handling structured student data, filtering, and preprocessing. NumPy is used for numerical operations and feature preparation before passing data to ML models.

d) Scikit-learn / TensorFlow (Machine Learning)

Scikit-learn is used to implement classification and regression models for recommendation and performance prediction. TensorFlow is considered for future deep learning extensions where larger or more complex models may be required.

e) Sentence-BERT / NLP Libraries

NLP libraries are used in text-based features such as chatbot interaction. Sentence-level embedding tools help in understanding user queries and returning more meaningful responses.

6.4 Tools and Platforms

The following development tools and platforms were used:

a) Visual Studio Code (VS Code)

VS Code was used as the primary IDE for coding, debugging, and project management. Its extensions and integrated terminal improved productivity during development.

b) Anaconda (if used)

Anaconda was used for managing Python environments and package dependencies. It helps avoid version conflicts and ensures stable execution of ML libraries.

c) Git and GitHub (Version Control)

Git was used to track code changes, and GitHub was used for repository hosting and collaboration. This made project management easier and maintained a clear development history.

6.5 Features of Software Used

The selected software stack offers several key advantages for this project:

a) Easy Development

The tools are beginner-friendly and support faster implementation with clear documentation and community support.

b) Scalability

The modular structure of backend services and frontend components allows future expansion without major redesign.

c) Cross-Platform Support

The software can run on different operating systems such as Windows, Linux, and macOS, making development and testing flexible.

d) Strong ML Integration

Python-based frameworks integrate directly with machine learning libraries, enabling smooth transition from data input to model output.

e) Maintainability

Clean code structure and component-based development improve readability and simplify future updates.

CHAPTER 7

PROJECT PLAN

7.1 PROJECT SCHEDULING

Timeline Chart:

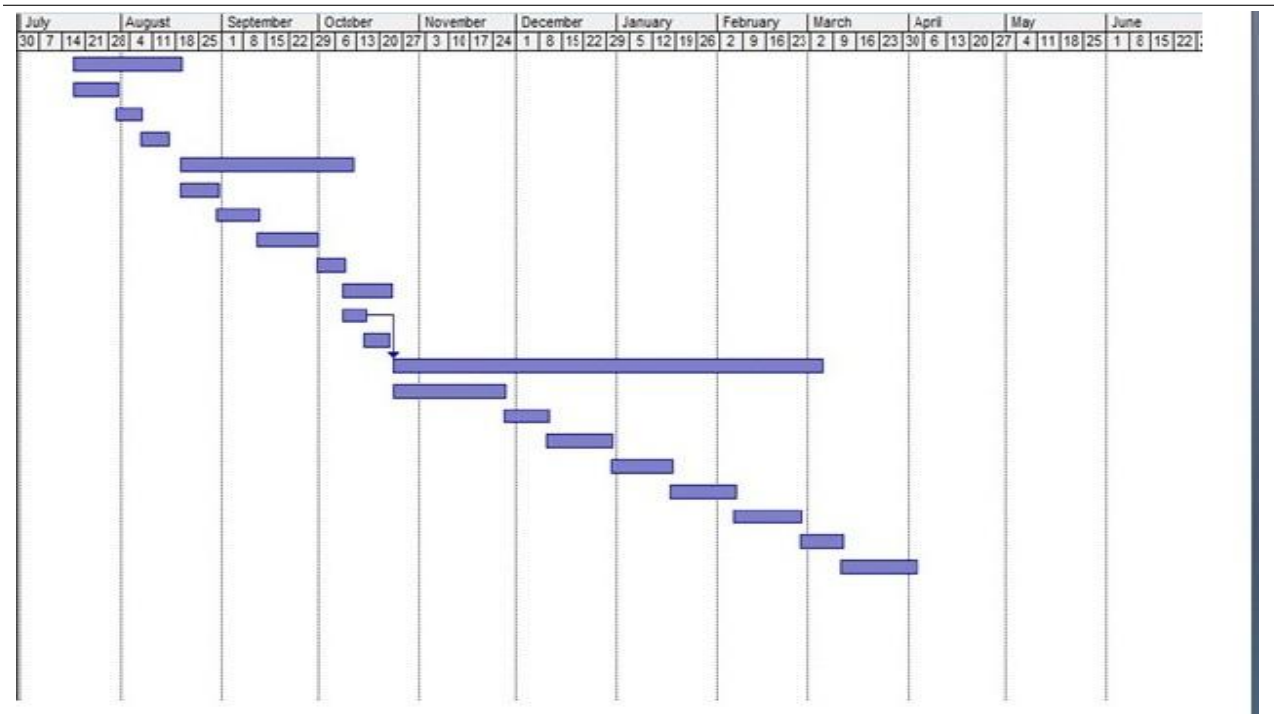


Figure 7.1: Timeline chart

Team Organization: Team consists of 4 members and proper planning mechanism are used and roles of each member are defined.

Team structure: The team structure for the project is identified. There are total 4 members in our team and roles are defined. All members are contributing in all the phases of project.

Table 2: Project Schedule

Month Scheduled	Phase	Work Done
12 June 2025 - 4 August 2025	Topic Searching	Topic Searched
4 August 2025 - 9 Sept 2025	Topic Selection	Topic Selected
4 August 2025 - 9 Sept 2025	Project Confirmation	Project Confirmed
4 August 2025 - 9 Sept 2025	Literature survey	Literature survey done
9 Sept 2025 - 29 Oct 2025	Requirement Analysis	Requirement Analysis done
9 Sept 2025 - 29 Oct 2025	Requirement Gathering	Requirement Gathered
3 Nov 2025 - 19 Dec 2025	Designing	Architecture Design
3 Nov 2025 - 19 Dec 2025	Designing Test	GUI Tested
3 Nov 2025 - 19 Dec 2025	Database Creation	Database Tested
5 Jan 2026 - 18 Feb 2026	Coding	Coded Different Modules
5 Jan 2026 - 18 Feb 2026	Database and modules connectivity	Connectivity done
5 Mar 2026 - 13 April 2026	Testing of project	Project Tested
13 April 2026 - 20 April 2026	Result Analysis	Result Analysis

Management reporting and communication: Well planning mechanisms are used for progress reporting and inter/intra team communication are identified as per requirements of the project.

7.2 PROJECT PLAN

NO	TASK	DURATION (Approx. Days)	START DATE	END DATE
1	Group Formation / Decide Area	53 Days	12/06/2025	04/08/2025
2	Topic Searching	53 Days	12/06/2025	04/08/2025
3	Topic Selection	36 Days	04/08/2025	09/09/2025
4	Project Confirmation	36 Days	04/08/2025	09/09/2025
5	Literature Survey	36 Days	04/08/2025	09/09/2025
6	Requirement Analysis	50 Days	09/09/2025	29/10/2025
7	Requirement Gathering	50 Days	09/09/2025	29/10/2025
8	Architecture Design (Designing)	46 Days	03/11/2025	19/12/2025
9	GUI Designing & Testing	46 Days	03/11/2025	19/12/2025

10	Database Creation & Testing	46 Days	03/11/2025	19/12/2025
11	Coding Different Modules	44 Days	05/01/2026	18/02/2026
12	Database and Modules Connectivity	44 Days	05/01/2026	18/02/2026
13	Testing of Project	39 Days	05/03/2026	13/04/2026
14	Result Analysis	7 Days	13/04/2026	20/04/2026
15	Final Documentation & Submission	Completed	23/04/2026	Completed

7.3 RESPONSIBILITIES OF DEVELOPER

The developer team was responsible for the complete lifecycle of the project, including:

- a) Requirement analysis and module definition
- b) System design (architecture, API flow, database, and UI planning)
- c) Backend and frontend development
- d) Machine learning/NLP module integration
- e) Testing and debugging of all modules
- f) Documentation and report preparation
- g) Post-development improvements and maintenance support

7.4 Overview of responsibilities of Developer

- a) Understand and analyze project requirements based on academic and user needs.
- b) Plan system architecture, database design, API structure, and module workflow.
- c) Develop frontend interfaces using React for smooth user interaction.
- d) Develop backend services using Flask for authentication, data flow, and API integration.
- e) Implement and integrate machine learning and NLP-based recommendation modules.
- f) Perform testing (unit, integration, and end-to-end) to ensure correctness and reliability.
- g) Fix bugs, optimize performance, and improve usability during development.
- h) Prepare technical documentation, report content, and project presentation materials.
- i) Maintain code quality, version control practices, and module-level consistency.
- j) Support future enhancements and minor maintenance after project completion.

7.5 Test Cases

Test Case ID	Test Case Description	Input	Expected Output	Status
TC01	Verify user login with valid credentials	Valid username and password	User logged in successfully	Pass
TC02	Verify user registration	Valid user details (name, email, password)	User registered successfully	Pass
TC03	Verify login with invalid credentials	Invalid username/password	Error message displayed	Pass
TC04	Input student academic details	Marks, subjects, interests	Data stored successfully	Pass
TC05	Validate missing input fields	Empty fields in form	Validation error shown	Pass
TC06	Train Random Forest model with dataset	Preprocessed student dataset	Model trained successfully	Pass

TC07	Verify prediction generation	Student data input	Recommended study plan generated	Pass
TC08	Check recommendation accuracy	Valid test dataset	Accurate recommendations displayed	Pass
TC09	Verify handling of invalid input data	Incorrect data format (text instead of numbers)	Error/validation message shown	Pass
TC10	Display recommended subjects/resources	Model output	List_of recommended subjects/materials shown	Pass
TC11	Verify performance for large dataset	Large dataset input	System processes without crash	Pass
TC12	Check model prediction speed	Student input data	Results generated within acceptable time	Pass
TC13	Verify system response for new user (no history)	New user data	Default/general recommendations provided	Pass
TC14	Update recommendations based on new data	Updated student performance	New recommendations generated	Pass
TC15	Verify UI display of recommendations	Generated output	Recommendations displayed clearly on dashboard	Pass

CHAPTER 8

OTHER SPECIFICATION

8.1 ADVANTAGES OF SYSTEM

a) Interactive and Responsive User Experience (React)

React provides a smooth and fast interface for login, dashboard, profile updates, and recommendation views.

Users can access results without full page reloads, which improves usability and engagement.

b) Lightweight and Efficient Backend Services (Flask)

Flask enables clean REST API development for authentication, recommendation, prediction, and progress modules.

Its modular design makes backend logic easy to maintain and extend for future features.

c) Data-Driven Personalized Recommendations (Machine Learning)

ML models analyze student performance, study consistency, and interests to generate personalized topic suggestions.

This improves the relevance of recommendations compared to one-size-fits-all study guidance.

d) Intelligent Query Understanding (NLP-Based Recommendation)

NLP components help the system understand user text input and learning intent more effectively.

This allows better matching of recommendations and chatbot responses to user needs.

e) Faster Academic Decision Support

The system automates profiling, analysis, and recommendation generation in a single workflow. Students and mentors receive actionable insights quickly, reducing manual analysis time.

f) Scalable Full-Stack Architecture

React frontend, Flask backend, and ML/NLP services can be scaled independently based on usage load.

This supports growth from small classroom use to broader institutional deployment.

g) Technology Integration for Future Expansion

The selected stack supports easy integration with analytics dashboards, LMS platforms, and mobile clients.

It creates a strong base for adding advanced models and real-time adaptive learning features.

8.2 APPLICATIONS

a) University Student Learning Portals

The system can be integrated into college portals to provide personalized study plans and topic recommendations.

It helps students improve weak subjects using performance-based guidance.

b) E-Learning Websites and MOOC Platforms

Online learning platforms can use this system to recommend next topics, practice modules, and revision paths.

This improves course completion rates through adaptive support.

c) Mobile Learning Applications

React-based frontend logic and Flask APIs can be extended to mobile apps for anywhere-access learning support.

Students can receive recommendations, predictions, and alerts in real time.

d) Coaching and Test Preparation Institutes

Coaching centers can use ML/NLP recommendations to assign targeted practice based on student profiles.

This enables structured preparation for competitive exams with data-backed planning.

e) Academic Mentoring and Counseling Systems

Mentors can use dashboard outputs to track learner progress and provide focused academic advice.

The system supports evidence-based counseling instead of subjective judgment alone.

f) Self-Study Assistance Platforms

Independent learners can use the system to identify weak areas, track consistency, and plan revision cycles.

NLP-based interaction also helps users ask for recommendations in natural language.

g) Institutional Performance Monitoring Tools

Educational institutions can use aggregated outputs for identifying learning trends and intervention needs. This supports better academic planning at both student and class levels.

CHAPTER 9

RESULTS AND DISCUSSION

9.1 RESULTS

The developed AI Study Recommendation System produced meaningful outputs for the main functional modules, including authentication, student profiling, recommendation generation, performance prediction, and result visualization. When a user opens the system, the first interaction is the login process. After successful login, the user is redirected to the dashboard, where personal study information and progress-related data are shown. This confirms that the system correctly handles user authentication and session-based access to protected pages.

After login, the user provides academic profile details such as interests, skill level, completed tasks, and performance-related values. The backend processes this input and prepares it for prediction and recommendation modules. Once the data is submitted, the machine learning pipeline evaluates topic strength and expected performance trend. Based on these outputs, the system generates personalized recommendations, such as which topics need higher attention, what type of study plan should be followed, and where practice effort should be increased.

The output is displayed clearly on the user interface through dashboard sections and recommendation pages. The user can view predicted performance values, topic-level status, and suggested actions for improvement. In addition, planner-related outputs guide the user toward structured study routines. During testing, the system produced stable outputs for repeated valid inputs and maintained proper flow between frontend and backend modules. From a performance perspective, response behavior was suitable for a final-year project prototype, with practical prediction and recommendation generation in normal usage conditions. Since the model is trained on a limited project dataset, the current results are best interpreted as functional and directionally useful rather than large-scale production-level accuracy claims.

9.1.1 DISCUSSION

The obtained results show that the system achieves its primary goal: providing personalized academic support based on student data. The workflow from login to recommendation output is complete and logically connected, and users can move through all major modules without manual intervention. This indicates that the integration of frontend pages, backend APIs, and machine learning services has been successfully implemented.

Compared with traditional systems that provide the same study material to all students, this project offers adaptive suggestions based on individual profile and progress values. This is an important advantage because it supports targeted learning rather than generic guidance. Another strength is the modular design, which makes the system easier to maintain and extend. Separate services for recommendation, prediction, planner, and progress tracking improve clarity and future scalability.

At the same time, some limitations were observed. The model uses a relatively small and controlled dataset, so broader validation on diverse real-world student populations is still required. The recommendation quality is useful for guidance, but not yet sufficient for fully automated high-stakes academic decisions. In addition, advanced explainability and richer visual analytics can be improved in future versions. Overall, the project delivers realistic and academically valid outcomes for a B.Tech-level implementation, while also identifying clear directions for enhancement.

CHAPTER 10

CONCLUSION

CONCLUSION

The AI Study Recommendation System successfully demonstrates how machine learning can be used to improve personalized learning support for students. The project combines user profile data, progress tracking, and predictive modeling to generate topic-level recommendations and performance insights. The developed system provides a practical workflow where students can log in, view their learning status, and receive focused study guidance based on their strengths and weak areas.

From an implementation perspective, the integration of frontend, backend APIs, and ML modules was completed in a structured and scalable manner. The system is user-friendly, modular, and suitable for academic environments. Overall, the project meets its objective of building an intelligent recommendation platform that can assist students in better planning and improving their study performance.

FUTURE SCOPE

- a) Introduce deep learning-based recommendation models for higher accuracy on larger datasets.
- b) Add real-time adaptive testing, where question difficulty changes automatically after each response.
- c) Integrate advanced NLP-based academic chatbot support for topic explanation and doubt solving.
- d) Include learning analytics dashboards for teachers to monitor class-level and student-level progress.
- e) Support multi-language content recommendations to improve accessibility for diverse learners.
- f) Add mobile application support for better usability and continuous student engagement.
- g) Improve explainability by showing why a topic or study plan was recommended.
- h) Deploy the system on cloud infrastructure for large-scale institutional use.

REFERENCES

- [1] Muhammad Ramazan and Adnan Abid, "A Novel Hybrid Approach for Driver Drowsiness Detection," *vol. 12, IEEE, 2024*
- [2] Ahmet Kolus, "A Systematic Review on Driver Drowsiness Detection Using Eye Activity Measures," *vol. 12, IEEE, Jul. 2024.*
- [3] Hamza Ahmad Madni, Ali Raza, Rukhshanda Sehar, Nisrean Thalji, and Laith Abualigah, "Novel Transfer Learning Approach for Driver Drowsiness Detection Using Eye Movement Behavior," *vol. 12, IEEE, 2024.*
- [4] Ahmet Kolus, "A Systematic Review on Driver Drowsiness Detection Using Eye Activity Measures," *vol. 12, IEEE, Jul. 2024.*
- [5] Maria Gonzalez, David Perez, and Laura Smith, "AI-Based Adaptive Learning Systems for Personalized Education," *vol. 12, IEEE, 2024.*
- [6] Koichi Fujiwara, Hiroki Iwamoto, Kentaro Hori, and Manabu, "Driver Drowsiness Detection Using R-R Interval of Electrocardiogram and Self-Attention Autoencoder," *vol. 9, no. 1, IEEE, Jan. 2024.*
- [7] Xinyu Chen and Yeqing Li, "Personalized Learning Recommendation System Based on Machine Learning Algorithms," *vol. 13, IEEE, 2023.*
- [8] S. K. Sinha and R. K. Gupta, "An Intelligent Student Performance Analysis and Recommendation System Using Data Mining Techniques," *vol. 11, IEEE, 2022.*
- [9] Linlin Zhang, Hideo Saito, Liang Yang, and Jiajie Wu, "Privacy-Preserving Federated Transfer Learning for Driver Drowsiness Detection," *Biol. Psychol., vol. 10, IEEE, 2022.*
- [10] Mayman Altameen, Ankit Kumar, Ramesh Chandra Poonia, Sandeep Kumar, and Abdul Khader Jilani Saudagar, "Early Identification and Detection of Driver Drowsiness by Hybrid Machine Learning," *vol. 9, IEEE, Nov. 2021.*
- [11] Ahmed Hassan and Noor Ul Amin, "A Collaborative Filtering Approach for Course Recommendation in E-Learning Platforms," *vol. 10, IEEE, 2021.*
- [12] Federico Guede-Fernandez and Mireya Fernandez-Chimeno, "Driver Drowsiness Detection Based on Respiratory Signal Analysis," *vol. 7, IEEE, Jul. 2019.*